接口文档：学生选课系统后端API

基础信息

- **基础URL**:localhost:8080
- **统一响应格式**: 使用 `Result` 对象封装，包含 `code` (1成功/0失败)， `msg` (描述信息)， `data` (具体数据)。
- **日期时间格式**: 如无特殊说明， `createTime` 和 `updateTime` 字段格式为 `yyyy-MM-dd'T'HH:mm:ss` (LocalDateTime 的默认JSON序列化格式)。

第一部分：用户 (User) 操作接口

1. 添加用户

- **请求方法**: POST
- **请求URL**: `/user`
- **请求参数**:
  - **位置**: 请求体 (RequestBody)
  - **类型**: application/json
  - **说明**: 一个完整的 `User` 对象JSON，无需传递`id`，`createTime`，`updateTime`（通常由后端自动生成）。
- **返回示例** (成功):

```
{
  "code": 1,
  "msg": "success",
  "data": {
    "id": 1001,
    "username": "zhangsan",
    "password": "已加密的密码",
    "name": "张三",
    "role": "student",
    "major": "计算机科学",
    "createTime": "2024-05-20T10:30:00",
    "updateTime": "2024-05-20T10:30:00"
  }
}
```

2. 根据ID查询用户

- **请求方法**: GET
- **请求URL**: `/user/{id}`
- **请求参数**:
  - **位置**: URL路径 (Path Variable)
  - **名称**: `id`
  - **说明**: 用户的唯一标识ID。
- **返回示例** (成功):

```
{
  "code": 1,
  "msg": "success",
  "data": {
    "id": 1001,
    "username": "zhangsan",
    "password": "已加密的密码",
    "name": "张三",
    "role": "student",
    "major": "计算机科学",
    "createTime": "2024-05-20T10:30:00",
    "updateTime": "2024-05-20T10:30:00"
  }
}
```

3. 条件分页查询用户列表

- **请求方法**: GET
- **请求URL**: `/user`
- **请求参数**:
  - **位置**: URL查询字符串 (Query String, 即 `?` 后面的参数)
  - **参数列表**:

```
参数名        必需  默认值  说明
page          否     1      当前页码
pageSize      否     10     每页记录数
name          否     -      用户姓名，支持模糊匹配
role          否     -      用户角色，如 `teacher`, `student`
major         否     -      用户专业
```

- **返回示例** (成功):

```
{
  "code": 1,
  "msg": "success",
  "data": {
    "totals": 85,
    "rows": [
      {
        "id": 1001,
        "username": "zhangsan",
        "password": "已加密的密码",
        "name": "张三",
        "role": "student",
        "major": "计算机科学",
        "createTime": "2024-05-20T10:30:00",
        "updateTime": "2024-05-20T10:30:00"
      },
      {
        "id": 1002,
        "username": "lisi",
        "password": "已加密的密码",
        "name": "李四",
        "role": "teacher",
        "major": "软件工程",
        "createTime": "2024-05-20T11:00:00",
        "updateTime": "2024-05-20T11:00:00"
      }
      // ... 最多 pageSize 条记录
    ]
  }
}
```

4. 修改用户信息

- **请求方法**: PUT
- **请求URL**: `/user`
- **请求参数**:
  - **位置**: 请求体 (RequestBody)
  - **类型**: application/json
  - **说明**: 一个包含更新后信息的完整（或部分）`User` 对象JSON，必须包含 `id` 字段以标识要修改的用户。
- **返回示例** (成功):

```
{
  "code": 1,
```

```
  "msg": "success",
  "data": null
}
```

5. 批量删除用户

- **请求方法**: DELETE
- **请求URL**: `/user/{ids}`
- **请求参数**:
  - **位置**: URL路径 (Path Variable)
  - **名称**: `ids`
  - **说明**: 要删除的用户ID列表，多个ID用英文逗号 `,` 分隔。例如：`/user/1001,1002,1003`
- **返回示例** (成功):

```
{
  "code": 1,
  "msg": "success",
  "data": null
}
```

第二部分：课程 (Course) 操作接口

1. 添加课程

- **请求方法**: POST
- **请求URL**: `/course`
- **请求参数**:
  - **位置**: 请求体 (RequestBody)
  - **类型**: application/json
  - **说明**: 一个完整的 `Course` 对象JSON，无需传递 `id`，`selected`，`createTime`，`updateTime`。
- **返回示例** (成功):

```
{
  "code": 1,
  "msg": "success",
  "data": {
    "id": 5001,
    "courseCode": "CS101",
    "courseName": "数据结构",
    "teacherId": "2001",
    "credit": 3.5,
```

```
      "capacity": 100,
      "selected": 0,
      "createTime": "2024-05-20T10:30:00",
      "updateTime": "2024-05-20T10:30:00"
    }
  }
```

2. 根据ID查询课程

- **请求方法**: GET
- **请求URL**: `/course/{id}`
- **请求参数**:
  - **位置**: URL路径 (Path Variable)
  - **名称**: `id`
  - **说明**: 课程的唯一标识ID。
- **返回示例** (成功):

```
{
  "code": 1,
  "msg": "success",
  "data": {
    "id": 5001,
    "courseCode": "CS101",
    "courseName": "数据结构",
    "teacherId": "2001",
    "credit": 3.5,
    "capacity": 100,
    "selected": 45,
    "createTime": "2024-05-20T10:30:00",
    "updateTime": "2024-05-20T10:30:00"
  }
}
```

3. 条件分页查询课程列表

- **请求方法**: GET
- **请求URL**: `/course`
- **请求参数**:
  - **位置**: URL查询字符串 (Query String)
  - **参数列表**:

```
参数名          必需   默认值   说明
page             否      1       当前页码
```

```
pageSize        否     10     每页记录数
courseCode      否     -      课程代码，支持模糊匹配
courseName      否     -      课程名称，支持模糊匹配
teacherId       否     -      授课教师ID
minCredit       否     0      学分数下限
maxCredit       否     10     学分数上限
minCapacity     否     0      课程容量下限
maxCapacity     否     1000   课程容量上限
isRemaining     否     -      是否只查询有余量(`capacity` > `selected`)的课程。传 `true` 时生效。
```

- **返回示例** (成功，查询有余量的课程):

```
{
  "code": 1,
  "msg": "success",
  "data": {
    "totals": 23,
    "rows": [
      {
        "id": 5001,
        "courseCode": "CS101",
        "courseName": "数据结构",
        "teacherId": "2001",
        "credit": 3.5,
        "capacity": 100,
        "selected": 45,
        "createTime": "2024-05-20T10:30:00",
        "updateTime": "2024-05-20T10:30:00"
      },
      {
        "id": 5002,
        "courseCode": "MATH201",
        "courseName": "高等数学",
        "teacherId": "2002",
        "credit": 4.0,
        "capacity": 150,
        "selected": 120,
        "createTime": "2024-05-20T11:00:00",
        "updateTime": "2024-05-20T11:00:00"
      }
    ]
  }
}
```

4. 修改课程信息

- **请求方法**: PUT
- **请求URL**: `/course`
- **请求参数**:
  - **位置**: 请求体 (RequestBody)
  - **类型**: application/json
  - **说明**: 一个包含更新后信息的完整（或部分）`Course` 对象JSON，必须包含 `id` 字段以标识要修改的课程。
- **返回示例** (成功):

```
{
  "code": 1,
  "msg": "success",
  "data": null
}
```

5. 批量删除课程

- **请求方法**: DELETE
- **请求URL**: `/course/{ids}`
- **请求参数**:
  - **位置**: URL路径 (Path Variable)
  - **名称**: `ids`
  - **说明**: 要删除的课程ID列表，多个ID用英文逗号 `,` 分隔。例如：`/course/5001,5002`
- **返回示例** (成功):

```
{
  "code": 1,
  "msg": "success",
  "data": null
}
```

（注：文档部分内容可能由 AI 生成）